Principles of Programming Languages
Lecture 12: Paradigms: Logic Programming.

Andrei Arusoaie[1]

[1]Department of Computer Science

January 8, 2017

Outline

Outline

Paradigms

- Imperative Programming: ✓

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming: ✓

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming: ✓
- Functional Programming: ✓

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming: ✓
- Functional Programming: ✓
- Logic Programming

General facts about PLs

General facts about PLs

We have seen PLs where the computations are based on:

General facts about PLs

We have seen PLs where the computations are based on:

- *state* + *modifiable variable* + *assignment*

General facts about PLs

We have seen PLs where the computations are based on:

- *state* + *modifiable variable* + *assignment*
- higher-order + recursion

General facts about PLs

We have seen PLs where the computations are based on:

- *state* + *modifiable variable* + *assignment*
- higher-order + recursion

Are there other approaches?

General facts about PLs

We have seen PLs where the computations are based on:

- *state* + *modifiable variable* + *assignment*
- higher-order + recursion

Are there other approaches?

- Yes

General facts about PLs

We have seen PLs where the computations are based on:

- *state* + *modifiable variable* + *assignment*
- higher-order + recursion

Are there other approaches?

- Yes: Logic programming

Some background

Some background

- Computation vs. Deduction

Some background

- Computation vs. Deduction
  - Computation: 10 + 0 = 10 (using some predefined rules, i.e., $n + 0 = 0$)

Some background

- Computation vs. Deduction
  - Computation: 10 + 0 = 10 (using some predefined rules, i.e., $n + 0 = 0$)
  - Deduction: requires more than automatic application of rules

Some background

- Computation vs. Deduction
  - Computation: 10 + 0 = 10 (using some predefined rules, i.e., $n + 0 = 0$)
  - Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete

Some background

- Computation vs. Deduction
  - Computation: 10 + 0 = 10 (using some predefined rules, i.e., $n + 0 = 0$)
  - Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete
  - Example: $a^2 + b^2 = c^2$

Some background

- Computation vs. Deduction
  - Computation: 10 + 0 = 10 (using some predefined rules, i.e., $n + 0 = 0$)
  - Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete
  - Example: $a^2 + b^2 = c^2$ ... Pythagoras $\rightarrow$ first proof

Some background

Some background

Reminder:

Some background

Reminder:

- Terms: $f(t_1, \ldots, t_n)$ – where f has arity n, and can contain variables

Some background

Reminder:

- Terms: $f(t_1, \ldots, t_n)$ – where f has arity n, and can contain variables
- Predicates: $p(t_1, \ldots, t_n)$ – where p is a predicate of arity n

Some background

Reminder:

- Terms: $f(t_1, \ldots, t_n)$ – where f has arity n, and can contain variables
- Predicates: $p(t_1, \ldots, t_n)$ – where p is a predicate of arity n
- Inference rules:

$$\frac{C_1 \quad C_2 \quad \ldots \quad C_n}{H} \text{ RULENAME}$$

Some background

Reminder:

- Terms: $f(t_1, \ldots, t_n)$ – where f has arity n, and can contain variables
- Predicates: $p(t_1, \ldots, t_n)$ – where p is a predicate of arity n
- Inference rules:

$$\frac{C_1 \quad C_2 \quad \ldots \quad C_n}{H}\ \textsc{RuleName}$$

- Examples:

$$\frac{}{nat(0)}\ \textsc{Zero} \qquad \frac{nat(N)}{nat(s(N))}\ \textsc{Succ}$$

Some background

Reminder:

- Terms: $f(t_1, \ldots, t_n)$ – where f has arity n, and can contain variables
- Predicates: $p(t_1, \ldots, t_n)$ – where p is a predicate of arity n
- Inference rules:

$$\frac{C_1 \quad C_2 \quad \ldots \quad C_n}{H}\ \textsc{RuleName}$$

- Examples:

$$\frac{}{nat(0)}\ \textsc{Zero} \qquad \frac{nat(N)}{nat(s(N))}\ \textsc{Succ}$$

$$\frac{}{even(0)}\ \textsc{Even0} \qquad \frac{even(N)}{even(s(s(N)))}\ \textsc{EvenSucc}$$

Proofs

- Proof that $even(s(s(s(s(0)))))$ holds:

$$\dfrac{\dfrac{\dfrac{\dfrac{\cdot}{even(0)}\,\textsc{Even0}}{even(s(s(0)))}\,\textsc{EvenSucc}}{even(s(s(s(s(0))))))}\,\textsc{EvenSucc}}$$

Proofs

- Proof that $even(s(s(s(s(0)))))$ holds:

$$\dfrac{\dfrac{\dfrac{\dfrac{\cdot}{even(0)}\,\textsc{Even0}}{even(s(s(0)))}\,\textsc{EvenSucc}}{even(s(s(s(s(0))))))}\,\textsc{EvenSucc}}$$

- Proof strategies: *goal-directed* vs. *forward-reasoning*

Proofs

- Proof that $even(s(s(s(s(0)))))$ holds:

$$\dfrac{\dfrac{\dfrac{\dfrac{\cdot}{even(0)}\,\textsc{Even0}}{even(s(s(0)))}\,\textsc{EvenSucc}}{even(s(s(s(s(0))))))}\,\textsc{EvenSucc}$$

- Proof strategies: *goal-directed* vs. *forward-reasoning*
- Logic programming uses *goal-directed*: for the current goal, search the rule that might have been applied to arrive at this conclusion

$$\dfrac{}{even(0)}\,\textsc{Even0} \qquad \dfrac{even(N)}{even(s(s(N)))}\,\textsc{EvenSucc}$$

Logic programming: general facts

- This paradigm is based on formal logic

Logic programming: general facts

- This paradigm is based on formal logic
- Programs are sets of sentences in the logical form of clauses (referred to as rules):

Logic programming: general facts

- This paradigm is based on formal logic
- Programs are sets of sentences in the logical form of clauses (referred to as rules):
  - $H :- C_1, \ldots, C_2.$

Logic programming: general facts

- This paradigm is based on formal logic
- Programs are sets of sentences in the logical form of clauses (referred to as rules):
  - $H :- C_1, \ldots, C_2.$
  - H is the *head* of the clause

Logic programming: general facts

- This paradigm is based on formal logic
- Programs are sets of sentences in the logical form of clauses (referred to as rules):
  - $H :- C_1, \ldots, C_2.$
  - H is the *head* of the clause
  - $C_1, \ldots, C_2$ is the *body*

Logic programming: general facts

- This paradigm is based on formal logic
- Programs are sets of sentences in the logical form of clauses (referred to as rules):
  - $H :- C_1, \ldots, C_2.$
  - H is the *head* of the clause
  - $C_1, \ldots, C_2$ is the *body*
  - $H.$ denotes a clause without body, a.k.a. *facts*

Logic programming: general facts

- This paradigm is based on formal logic
- Programs are sets of sentences in the logical form of clauses (referred to as rules):
  - $H :- C_1, \ldots, C_2.$
  - H is the *head* of the clause
  - $C_1, \ldots, C_2$ is the *body*
  - $H.$ denotes a clause without body, a.k.a. *facts*
- Logic programming can be seen as controlled deduction

Logic programming: general facts

- This paradigm is based on formal logic
- Programs are sets of sentences in the logical form of clauses (referred to as rules):
  - $H :- C_1, \ldots, C_2.$
  - H is the *head* of the clause
  - $C_1, \ldots, C_2$ is the *body*
  - $H.$ denotes a clause without body, a.k.a. *facts*
- Logic programming can be seen as controlled deduction
- Logic programming languages (or families of PL): Prolog, ASP (Answer set programming), Datalog

Logic programming: how it works

- Demo in Prolog: goals.

$$\frac{}{nat(0)}\ \textsc{Zero} \qquad \frac{nat(N)}{nat(s(N))}\ \textsc{Succ}$$

Logic programming: how it works

- Demo in Prolog: goals.

$$\frac{}{nat(0)}\ \textsc{Zero} \qquad \frac{nat(N)}{nat(s(N))}\ \textsc{Succ}$$

$$\frac{}{even(0)}\ \textsc{Even0} \qquad \frac{even(N)}{even(s(s(N)))}\ \textsc{EvenSucc}$$

Answer substitutions

- Is it possible to use variables in goals, i.e, $even(s(s(X))$?

Answer substitutions

- Is it possible to use variables in goals, i.e, $even(s(s(X))$?
  Yes: use unification.

Answer substitutions

- Is it possible to use variables in goals, i.e, $even(s(s(X))$? Yes: use unification.
- Demo in Prolog: queries.

$$\frac{}{even(0)}\ \textsc{Even0} \qquad \frac{even(N)}{even(s(s(N)))}\ \textsc{EvenSucc}$$

Answer substitutions

- Is it possible to use variables in goals, i.e, $even(s(s(X))$? Yes: use unification.
- Demo in Prolog: queries.

$$\frac{}{even(0)}\ \textsc{Even0} \qquad \frac{even(N)}{even(s(s(N)))}\ \textsc{EvenSucc}$$

Other aspects:

Answer substitutions

- Is it possible to use variables in goals, i.e, $even(s(s(X))$? Yes: use unification.
- Demo in Prolog: queries.

$$\frac{}{even(0)}\ \textsc{Even0} \qquad \frac{even(N)}{even(s(s(N)))}\ \textsc{EvenSucc}$$

Other aspects:

- Backtracking: more than one possible rule applicable

Answer substitutions

- Is it possible to use variables in goals, i.e, $even(s(s(X))$? Yes: use unification.
- Demo in Prolog: queries.

$$\frac{}{even(0)}\ \textsc{Even0} \qquad \frac{even(N)}{even(s(s(N)))}\ \textsc{EvenSucc}$$

Other aspects:

- Backtracking: more than one possible rule applicable
- Subgoal order: which of the premises to consider?

Demo

Prolog demo: light meal problem

Resources

Reading material:

- Chapter/Lecture 1: `https://people.mpi-sws.org/~dg/teaching/lis2014/modules/lp-fp-07.pdf`